

Trạng thái	Đã xong
Bắt đầu vào lúc	Thứ Bảy, 3 tháng 8 2024, 10:46 AM
Kết thúc lúc	Thứ Sáu, 23 tháng 8 2024, 3:52 PM
Thời gian thực hiện	20 Các ngày 5 giờ
Điểm	3,00/3,00
Điểm	10,00 trên 10,00 (100%)



Câu hỏi 1

Đúng

Đạt điểm 1,00 trên 1,00

Given class SplayTree definition:



```

class SplayTree {
    struct Node {
        int val;
        Node* pLeft;
        Node* pRight;
        Node* pParent;
        Node(int val = 0, Node* l = nullptr, Node* r = nullptr, Node* par = nullptr) : val(val), pLeft(l), pRight(r),
        pParent(par) { }
    };
    Node* root;

    // print the tree structure for local testing
    void printBinaryTree(string prefix, const Node* root, bool isLeft, bool hasRightSibling) {
        if (!root && isLeft && hasRightSibling) {
            cout << prefix << "├─\n";
        }
        if (!root) return;
        cout << prefix;
        if (isLeft && hasRightSibling)
            cout << "├─";
        else
            cout << "└─";
        cout << root->val << '\n';
        printBinaryTree(prefix + (isLeft && hasRightSibling ? "| " : "  "), root->pLeft, true, root->pRight);
        printBinaryTree(prefix + (isLeft && hasRightSibling ? "| " : "  "), root->pRight, false, root->pRight);
    }

    void printPreorder(Node* p) {
        if (!p) {
            return;
        }
        cout << p->val << ' ';
        printPreorder(p->pLeft);
        printPreorder(p->pRight);
    }
public:
    SplayTree() {
        root = nullptr;
    }
    ~SplayTree() {
        // Ignore deleting all nodes in the tree
    }

    void printBinaryTree() {
        printBinaryTree("", root, false, false);
    }

    void printPreorder() {
        printPreorder(root);
        cout << "\n";
    }

    void splay(Node* p) {
        // To Do
    }

    void insert(int val) {
        // To Do
    }
};

```

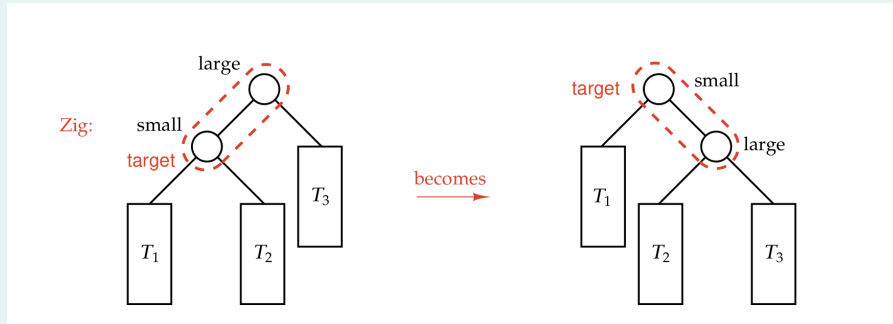
Implement the following method:

1. void splay(Node* p): bottom-up splaying a Node

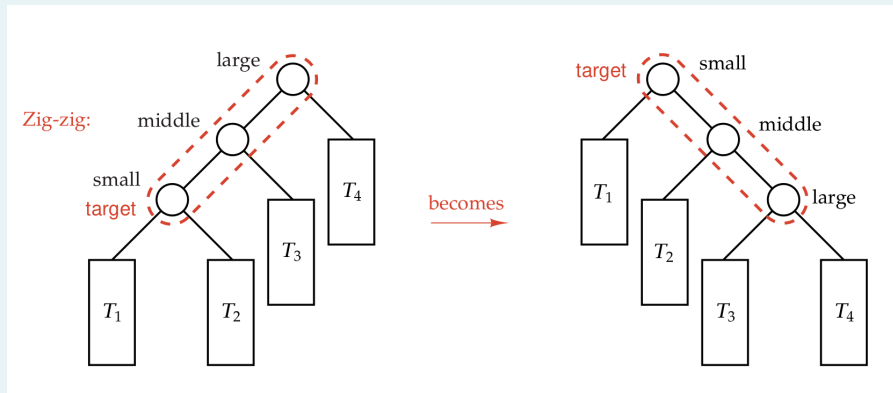
When a splay operation is performed on Node p, it will be moved to the root. To perform a splay operation we carry out a sequence of splay steps, each of which moves p closer to the root.

The three types of splay steps are:

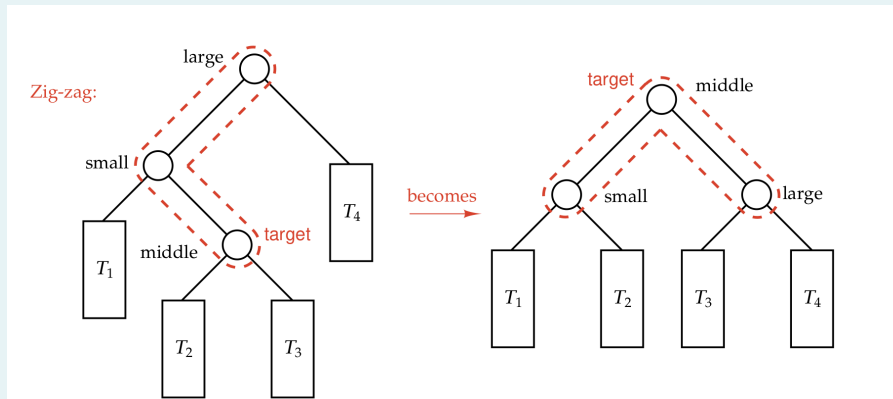
- Zig step



- Zig-zig step:



- Zig-zag step:



Note: there are also zag, zag-zag and zag-zig step but we don't show them here

2. void insert(int val):

To insert a value val into a splay tree:

- + Insert val as with a normal [binary search tree](#).
- + When the new value is inserted, a splay operation is performed. As a result, the newly inserted node becomes the root of the tree.

Note: In a splay tree, the values in the left subtree $\leq$ root's value $\leq$ the values in the right subtree. In this exercise, when inserting a duplicate value, you have to insert it to the right subtree to pass the testcases.

Constraint of testcases:

+ number of operation $\leq 10^4$

+ $1 \leq \text{val} \leq 10^5$

For example:

Test	Input	Result
<pre>SplayTree tree; int query; cin >> query; for(int i = 0; i < query; i++) { string op; int val; cin >> op >> val; if (op == "insert") tree.insert(val); } // print preorder traversal of the tree tree.printPreorder(); // print structure of the tree tree.printBinaryTree();</pre>	<pre>6 insert 50 insert 70 insert 30 insert 80 insert 100 insert 90</pre>	<pre>90 80 30 70 50 100 └─90 └─80 └─30 └─70 └─50 └─100</pre>
<pre>SplayTree tree; int query; cin >> query; for(int i = 0; i < query; i++) { string op; int val; cin >> op >> val; if (op == "insert") tree.insert(val); } // print preorder traversal of the tree tree.printPreorder(); // print structure of the tree tree.printBinaryTree();</pre>	<pre>6 insert 95 insert 200 insert 80 insert 100 insert 200 insert 95</pre>	<pre>95 95 80 200 100 200 └─95 └─95 └─80 └─200 └─100 └─200</pre>

Answer: (penalty regime: 0 %)

Reset answer

```
1 void rotateLeft(Node* x) {
2     Node* y = x->pRight;
3     x->pRight = y->pLeft;
4     if (y->pLeft != nullptr)
5         y->pLeft->pParent = x;
6     y->pParent = x->pParent;
7     if (x->pParent == nullptr)
8         root = y;
9     else if (x == x->pParent->pLeft)
10        x->pParent->pLeft = y;
11    else
12        x->pParent->pRight = y;
13    y->pLeft = x;
14    x->pParent = y;
15 }
16
17 void rotateRight(Node* x) {
18     Node* y = x->pLeft;
19     x->pLeft = y->pRight;
20     if (y->pRight != nullptr)
21         y->pRight->pParent = x;
22     y->pParent = x->pParent;
23     if (x->pParent == nullptr)
24         root = y;
25     else if (x == x->pParent->pRight)
26         x->pParent->pRight = y;
27     else
28         x->pParent->pLeft = y;
```

```

28     x->pParent->pLeft = y,
29     y->pRight = x;
30     x->pParent = y;
31 }
32
33 void splay(Node* x) {
34     while (x->pParent != nullptr) {
35         if (x->pParent->pParent == nullptr) {
36             if (x->pParent->pLeft == x) {
37                 rotateRight(x->pParent);
38             } else {
39                 rotateLeft(x->pParent);
40             }
41         } else if (x->pParent->pLeft == x && x->pParent->pParent->pLeft == x->pParent) {
42             rotateRight(x->pParent->pParent);
43             rotateRight(x->pParent);
44         } else if (x->pParent->pRight == x && x->pParent->pParent->pRight == x->pParent) {
45             rotateLeft(x->pParent->pParent);
46             rotateLeft(x->pParent);
47         } else if (x->pParent->pLeft == x && x->pParent->pParent->pRight == x->pParent) {
48             rotateRight(x->pParent);
49             rotateLeft(x->pParent);
50         } else {
51             rotateLeft(x->pParent);
52             rotateRight(x->pParent);
53         }
54     }
55 }
56
57 void insert(int val) {
58     Node* z = root;
59     Node* p = nullptr;
60
61     while (z != nullptr) {
62         p = z;
63         if (val < z->val) {
64             z = z->pLeft;
65         } else {
66             z = z->pRight;
67         }
68     }
69
70     z = new Node(val);
71     z->pParent = p;
72
73     if (p == nullptr) {
74         root = z;
75     } else if (val < p->val) {
76         p->pLeft = z;
77     } else {
78         p->pRight = z;
79     }
80
81     splay(z);
82 }

```



	Test	Input	Expected	Got	
✓	<pre>SplayTree tree; int query; cin >> query; for(int i = 0; i < query; i++) { string op; int val; cin >> op >> val; if (op == "insert") tree.insert(val); } // print preorder traversal of the tree tree.printPreorder(); // print structure of the tree tree.printBinaryTree();</pre>	6 insert 50 insert 70 insert 30 insert 80 insert 100 insert 90	90 80 30 70 50 100 └─90 └─80 └─30 └─ └─70 └─50 └─100	90 80 30 70 50 100 └─90 └─80 └─30 └─ └─70 └─50 └─100	✓
✓	<pre>SplayTree tree; int query; cin >> query; for(int i = 0; i < query; i++) { string op; int val; cin >> op >> val; if (op == "insert") tree.insert(val); } // print preorder traversal of the tree tree.printPreorder(); // print structure of the tree tree.printBinaryTree();</pre>	6 insert 95 insert 200 insert 80 insert 100 insert 200 insert 95	95 95 80 200 100 200 └─95 └─95 └─80 └─200 └─100 └─ └─200	95 95 80 200 100 200 └─95 └─95 └─80 └─200 └─100 └─ └─200	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.



Câu hỏi 2

Đúng

Đạt điểm 1,00 trên 1,00

Given class SplayTree definition:

```
class SplayTree {
    struct Node {
        int val;
        Node* pLeft;
        Node* pRight;
        Node* pParent;
        Node(int val = 0, Node* l = nullptr, Node* r = nullptr, Node* par = nullptr) : val(val), pLeft(l), pRight(r),
        pParent(par) { }
    };
```

```
    Node* root;
```

```
// print the tree structure for local testing
void printBinaryTree(string prefix, const Node* root, bool isLeft, bool hasRightSibling) {
    if (!root && isLeft && hasRightSibling) {
        cout << prefix << "├─\n";
    }
    if (!root) return;
    cout << prefix;
    if (isLeft && hasRightSibling)
        cout << "├─";
    else
        cout << "└─";
    cout << root->val << '\n';
    printBinaryTree(prefix + (isLeft && hasRightSibling ? "| " : "  "), root->pLeft, true, root->pRight);
    printBinaryTree(prefix + (isLeft && hasRightSibling ? "| " : "  "), root->pRight, false, root->pRight);
}
```

```
void printPreorder(Node* p) {
    if (!p) {
        return;
    }
    cout << p->val << ' ';
    printPreorder(p->pLeft);
    printPreorder(p->pRight);
}
```

```
public:
    SplayTree() {
        root = nullptr;
    }
```

```
~SplayTree() {
    // Ignore deleting all nodes in the tree
}
```

```
void printBinaryTree() {
    printBinaryTree("", root, false, false);
}
```

```
void printPreorder() {
    printPreorder(root);
    cout << "\n";
}
```



```
void splay(Node* p);
```

```
void insert(int val);
```

```
bool search(int val) {  
    // To Do  
}  
};
```

Method splay and insert are already implemented

You have to implement the following method:

bool [search](#)(int val): [search](#) for the value val in the tree.

The [search](#) operation in splay tree do the same thing as BST [search](#). In addition, it also splays the node containing the value to the root.

- + If the [search](#) is successful, the node that is found will become the new root and the function return true.
- + Else, the last accessed node will be splayed and become the new root and the function return false.

Constraints of the testcases:

- + number of operation $\leq 10^4$
- + $1 \leq val \leq 10^5$

For example:

Test	Input	Result
<pre>SplayTree tree; int query; cin >> query; for(int i = 0; i < query; i++) { string op; int val; cin >> op >> val; if (op == "insert") tree.insert(val); else if (op == "search") cout << (tree.search(val) ? "found" : "not found") << '\n'; else if (op == "print") tree.printPreorder(); } tree.printBinaryTree();</pre>	<pre>8 insert 95 insert 200 insert 80 search 100 insert 55 insert 100 search 95 print 0</pre>	<pre>not found found 95 55 80 100 200 └─95 └─55 └─ └─80 └─100 └─200</pre>
<pre>SplayTree tree; int query; cin >> query; for(int i = 0; i < query; i++) { string op; int val; cin >> op >> val; if (op == "insert") tree.insert(val); else if (op == "search") cout << (tree.search(val) ? "found" : "not found") << '\n'; else if (op == "print") tree.printPreorder(); } tree.printBinaryTree();</pre>	<pre>5 insert 100 insert 200 insert 300 insert 200 search 250</pre>	<pre>not found └─300 └─200 └─200 └─100</pre>

Answer: (penalty regime: 0 %)

Reset answer

```
1
2
3 bool search(int val) {
4     Node* x = root;
5     Node* last = nullptr;
6
7     while (x != nullptr) {
8         last = x;
9         if (val == x->val) {
10             splay(x);
11             return true;
12         } else if (val < x->val) {
13             x = x->pLeft;
14         } else {
15             x = x->pRight;
16         }
17     }
18
19     if (last != nullptr) {
20         splay(last);
21     }
22
23     return false;
24 }
```

	Test	Input	Expected	Got	
✓	<pre>SplayTree tree; int query; cin >> query; for(int i = 0; i < query; i++) { string op; int val; cin >> op >> val; if (op == "insert") tree.insert(val); else if (op == "search") cout << (tree.search(val) ? "found" : "not found") << '\n'; else if (op == "print") tree.printPreorder(); } tree.printBinaryTree();</pre>	<pre>8 insert 95 insert 200 insert 80 search 100 insert 55 insert 100 search 95 print 0</pre>	<pre>not found found 95 55 80 100 200 └─95 └─55 └─ └─80 └─100 └─200</pre>	<pre>not found found 95 55 80 100 200 └─95 └─55 └─ └─80 └─100 └─200</pre>	✓

	Test	Input	Expected	Got	
✓	<pre>SplayTree tree; int query; cin >> query; for(int i = 0; i < query; i++) { string op; int val; cin >> op >> val; if (op == "insert") tree.insert(val); else if (op == "search") cout << (tree.search(val) ? "found" : "not found") << '\n'; else if (op == "print") tree.printPreorder(); } tree.printBinaryTree();</pre>	5 insert 100 insert 200 insert 300 insert 200 search 250	not found └─300 └─200 └─200 └─100	not found └─300 └─200 └─200 └─100	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.



Câu hỏi 3

Đúng

Đạt điểm 1,00 trên 1,00

Given class SplayTree definition:

```

class SplayTree {
    struct Node {
        int val;
        Node* pLeft;
        Node* pRight;
        Node* pParent;
        Node(int val = 0, Node* l = nullptr, Node* r = nullptr, Node* par = nullptr) : val(val), pLeft(l), pRight(r),
        pParent(par) { }
    };
    Node* root;

    // print the tree structure for local testing
    void printBinaryTree(string prefix, const Node* root, bool isLeft, bool hasRightSibling) {
        if (!root && isLeft && hasRightSibling) {
            cout << prefix << "├─\n";
        }
        if (!root) return;
        cout << prefix;
        if (isLeft && hasRightSibling)
            cout << "├─";
        else
            cout << "└─";
        cout << root->val << '\n';
        printBinaryTree(prefix + (isLeft && hasRightSibling ? "| " : "  "), root->pLeft, true, root->pRight);
        printBinaryTree(prefix + (isLeft && hasRightSibling ? "| " : "  "), root->pRight, false, root->pRight);
    }
}

```

```

void printPreorder(Node* p) {
    if (!p) {
        return;
    }
    cout << p->val << ' ';
    printPreorder(p->pLeft);
    printPreorder(p->pRight);
}

```

public:

```

SplayTree() {
    root = nullptr;
}

```

```

~SplayTree() {
    // Ignore deleting all nodes in the tree
}

```

```

void printBinaryTree() {
    printBinaryTree("", root, false, false);
}

```

```

void printPreorder() {
    printPreorder(root);
    cout << "\n";
}

```

```

void splay(Node* p);

```

```

void insert(int val);

```

```

bool search(int val);

```

```
Node* remove(int val) {
    // To Do
}
};
```

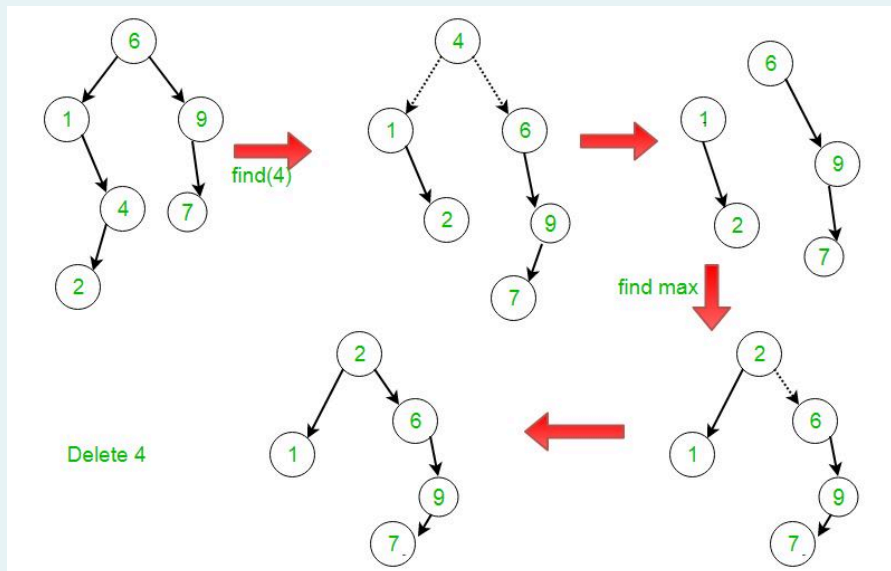
The methods `splay`, `insert` and [search](#) are already implemented.

Implement the following method:

`Node* remove(int val)`: remove the first Node with value equal to `val` from the tree and return it.

To perform remove operation on splay tree:

1. If root is NULL, return the root
2. [Search](#) for the first node containing the given value `val` and splay it. If `val` is present, the found node will become the root. Else the last accessed leaf node becomes the root.
3. If new root's value is not equal to `val`, return NULL as `val` is not present.
4. Else the value `val` is present, we remove root from the tree by the following steps:
 - 4.1 Split the tree into two tree: `tree1` = root's left subtree and `tree2` = root's right subtree
 - 4.2 If `tree1` is NULL, `tree2` is the new root
 - 4.3 Else, splay the leaf node with the largest value in `tree1`. `tree1` will be a left skewed [binary tree](#). Make `tree2` the right subtree of `tree1`. `tree1` becomes the new root
 - 4.4 Return the removed node.



Constraints of the testcases:

+ number of operations $\leq 10^4$

+ $1 \leq val \leq 10^5$

For example:



Test	Input	Result
<pre> SplayTree tree; int query; cin >> query; for(int i = 0; i < query; i++) { string op; int val; cin >> op >> val; if (op == "insert") tree.insert(val); else if (op == "remove") cout << (tree.remove(val) != nullptr ? "removed" : "not found") << '\n'; else if (op == "search") cout << (tree.search(val) ? "found" : "not found") << '\n'; else if (op == "print") tree.printPreorder(); } tree.printBinaryTree(); </pre>	<pre> 7 insert 100 insert 300 insert 200 insert 50 insert 250 remove 200 print 0 </pre>	<pre> removed 100 50 250 300 └─100 └─50 └─250 └─ └─300 </pre>
<pre> SplayTree tree; int query; cin >> query; for(int i = 0; i < query; i++) { string op; int val; cin >> op >> val; if (op == "insert") tree.insert(val); else if (op == "remove") cout << (tree.remove(val) != nullptr ? "removed" : "not found") << '\n'; else if (op == "search") cout << (tree.search(val) ? "found" : "not found") << '\n'; else if (op == "print") tree.printPreorder(); } tree.printBinaryTree(); </pre>	<pre> 7 insert 900 insert 1400 insert 100 insert 800 insert 750 remove 500 print 0 </pre>	<pre> not found 100 750 800 900 1400 └─100 └─ └─750 └─ └─800 └─ └─900 └─1400 </pre>
<pre> SplayTree tree; int query; cin >> query; for(int i = 0; i < query; i++) { string op; int val; cin >> op >> val; if (op == "insert") tree.insert(val); else if (op == "remove") cout << (tree.remove(val) != nullptr ? "removed" : "not found") << '\n'; else if (op == "search") cout << (tree.search(val) ? "found" : "not found") << '\n'; else if (op == "print") tree.printPreorder(); } </pre>	<pre> 12 insert 15 insert 3 remove 15 print 0 insert 5 insert 1 remove 1 insert 5 insert 9 insert 3 insert 13 print 0 </pre>	<pre> removed 3 removed 13 3 3 9 5 5 </pre>

Answer: (penalty regime: 0. %)

Reset answer

```

1 Node* getMax(Node* p){
2     while(p->pRight) p = p->pRight;
3     return p;
4 }
5 Node* remove(int val){
6     if(search(val)){
7         if(!this->root->pLeft){
8             Node* tempn = this->root;

```

```
9         this->root = this->root->pRight;
10         if(this->root) this->root->pParent = NULL;
11         return temp;
12     }
13     else{
14         Node* rightTree = this->root->pRight;
15         Node* leftTree = this->root->pLeft;
16         Node* rs = this->root;
17         this->root = leftTree;
18         this->root->pParent = NULL;
19         Node* p = getMax(this->root);
20         splay(p);
21         this->root->pRight = rightTree;
22         if(rightTree) rightTree->pParent = this->root;
23
24         return rs;
25     }
26 }
27 return NULL;
28 }
```

	Test	Input	Expected	Got	
✓	<pre>SplayTree tree; int query; cin >> query; for(int i = 0; i < query; i++) { string op; int val; cin >> op >> val; if (op == "insert") tree.insert(val); else if (op == "remove") cout << (tree.remove(val) != nullptr ? "removed" : "not found") << '\n'; else if (op == "search") cout << (tree.search(val) ? "found" : "not found") << '\n'; else if (op == "print") tree.printPreorder(); } tree.printBinaryTree();</pre>	<pre>7 insert 100 insert 300 insert 200 insert 50 insert 250 remove 200 print 0</pre>	<pre>removed 100 50 250 300 └─100 └─50 └─250 └─ └─300</pre>	<pre>removed 100 50 250 300 └─100 └─50 └─250 └─ └─300</pre>	✓



	Test	Input	Expected	Got	
✓	<pre>SplayTree tree; int query; cin >> query; for(int i = 0; i < query; i++) { string op; int val; cin >> op >> val; if (op == "insert") tree.insert(val); else if (op == "remove") cout << (tree.remove(val) != nullptr ? "removed" : "not found") << '\n'; else if (op == "search") cout << (tree.search(val) ? "found" : "not found") << '\n'; else if (op == "print") tree.printPreorder(); } tree.printBinaryTree();</pre>	<pre>7 insert 900 insert 1400 insert 100 insert 800 insert 750 remove 500 print 0</pre>	<pre>not found 100 750 800 900 1400 └─100 └─ └─750 └─ └─800 └─ └─900 └─ └─1400</pre>	<pre>not found 100 750 800 900 1400 └─100 └─ └─750 └─ └─800 └─ └─900 └─ └─1400</pre>	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.

